

Session-6

Question1: Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

Ans:

```
USE influencer_db;
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    product_name VARCHAR(100)  
);
```

```
INSERT INTO orders (user_id, product_name) VALUES
```

```
(1, 'Laptop'),
```

```
(1, 'Mouse'),
```

```
(1, 'Keyboard'),
```

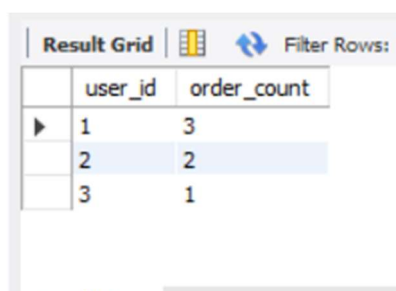
```
(2, 'Mobile'),
```

```
(2, 'Headphones'),
```

```
(3, 'Charger');
```

```
SELECT user_id, COUNT(*) AS order_count FROM orders
```

```
GROUP BY user_id;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. It contains a table with two columns: 'user_id' and 'order_count'. The table has three rows of data: user_id 1 with order_count 3, user_id 2 with order_count 2, and user_id 3 with order_count 1. The second row is highlighted in blue.

	user_id	order_count
▶	1	3
	2	2
	3	1

Question2: Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

Ans:

```
USE influencer_db;
```

```
CREATE TABLE restaurants (
```

```
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    name VARCHAR(100),
```

```
    cuisine VARCHAR(50),
```

```
    rating DECIMAL(2,1),
```

```
    city VARCHAR(50)
```

```
);
```

```
INSERT INTO restaurants (name, cuisine, rating, city) VALUES
```

```
('Spice Villa', 'Indian', 4.7, 'Rajkot'),
```

```
('Food Palace', 'Chinese', 3.8, 'Ahmedabad'),
```

```
('Royal Restaurant', 'Gujarati', 4.5, 'Rajkot'),
```

```
('Pizza House', 'Italian', 4.2, 'Surat'),
```

```
('Tasty Corner', 'Indian', 3.5, 'Jamnagar');
```

```
SELECT name, AVG(rating) AS average_rating FROM restaurants GROUP BY name
```

```
HAVING AVG(rating) > 4.0;
```



name	average_rating
Spice Villa	4.70000
Royal Restaurant	4.50000
Pizza House	4.20000

Question3: Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.

Ans:

```
USE influencer_db;
```

```
CREATE TABLE payments (
```

```
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    user_id INT,
```

```
    amount DECIMAL(10,2),
```

```
    payment_date DATE
```

```
);
```

```
INSERT INTO payments (user_id, amount, payment_date) VALUES
```

```
(1, 1500, '2026-09-01'),
```

```
(1, 2500, '2026-09-10'),
```

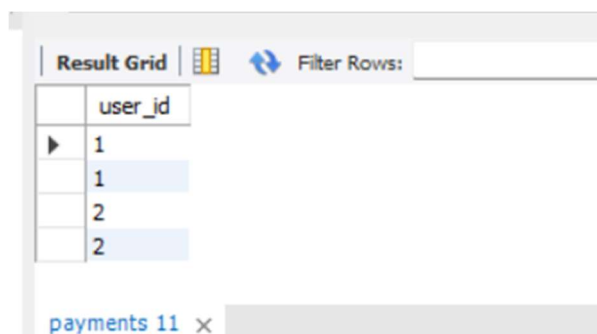
```
(2, 1800, '2026-09-05'),
```

```
(2, 3000, '2026-09-15'),
```

```
(3, 1200, '2026-09-20'),
```

```
(3, 1900, '2026-09-25');
```

```
SELECT user_id FROM payments WHERE user_id IN (SELECT user_id FROM payments  
WHERE amount > 2000);
```



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of a query, showing a single column 'user_id' with four rows. The first two rows contain the value '1' and the last two rows contain the value '2'. The rows are highlighted in a light blue color. At the bottom of the grid, there is a tab labeled 'payments 11' with a close button (x).

user_id
1
1
2
2

Question4: Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.

Ans:

```
USE influencer_db;
```

```
CREATE TABLE movies (  
    movie_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    rating DECIMAL(2,1)  
);
```

```
INSERT INTO movies (name, rating) VALUES
```

```
('KGF', 4.8),
```

```
('RRR', 4.5),
```

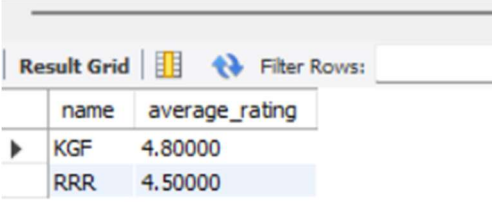
```
('Dangal', 4.2),
```

```
('Avatar', 3.9),
```

```
('Jawan', 3.7);
```

```
SELECT name, AVG(rating) AS average_rating FROM movies GROUP BY name
```

```
HAVING AVG(rating) > (SELECT AVG(rating) FROM movies);
```



	name	average_rating
▶	KGF	4.80000
	RRR	4.50000